Technology Stack

Phase 2: Requirement Analysis

1. Stack Overview

Trip Genius AI separates the browser interface from the API service. The frontend handles form inputs and presentation; the backend validates requests, coordinates AI generation and persistence, and returns structured data. A hosted model provides itinerary text, while Supabase stores trip information.

2. Frontend — React, TypeScript and Vite

React supports the component-based interface and state changes needed to collect inputs, display progress, show the generated itinerary and manage saved trips. TypeScript adds static type checking to help keep component data and API response handling consistent. Vite provides the development server and frontend build process. Styling is maintained in the frontend source CSS files.

3. Backend — Python and FastAPI

FastAPI exposes HTTP endpoints for health checks, itinerary generation, saved-trip operations and place searches. It provides request handling and can validate data using Pydantic models. Separating API routing from AI service logic makes it easier to debug errors and extend the application.

4. Generative AI — Groq

The AI service uses the Groq SDK and the configured model openai/gpt-oss-20b. The service builds a prompt from the user’s trip preferences and asks for structured itinerary output. Since a model response may not always be valid JSON or factually accurate, parsing and schema checks are important parts of the integration.

5. Data Validation — Pydantic

Pydantic models define the expected request and response structures. Validation makes the API contract more predictable and helps the frontend render the result consistently. Schema validation does not guarantee that every travel recommendation or cost is factually correct; it only checks structure and types.

6. Database — Supabase

Supabase provides the persistence layer for trip and itinerary records. The backend uses a configured client rather than connecting directly from the public browser to privileged database operations. Database schema and access policies should be reviewed before enabling multi-user production usage.

7. Hosting — Vercel and Render

The frontend is deployed on Vercel and the backend API on Render. Separate hosting allows the frontend and API to be updated independently, but it also requires correct API base URL and CORS configuration. The health endpoint can be used as a simple deployment smoke test.

8. External Discovery and Version Control

Google Maps search URLs are used for hotel and restaurant discovery. This approach avoids claiming direct access to live listing data but requires users to inspect the results themselves. Git and GitHub provide version history and a public location for the source repository. Secrets should not be committed.

9. Dependencies and Environment

The exact frontend dependencies are listed in frontend/package.json and backend Python dependencies are listed in backend/requirements.txt. Required credentials belong in private environment variables. When setting up a fresh environment, install dependencies from those files and confirm that the configured services are reachable before testing the complete flow.